4. Standard CNN architectures and transfer learning

March 7, 2026

[ ]:
```
# Mounting Drive
from google.colab import drive
drive.mount('/content/drive')
```

[ ]:
```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import warnings

# Suppress all warnings
warnings.filterwarnings('ignore')

# Define the paths
train_dir = '/content/drive/MyDrive/Deep Learning - datasets/alzheimer dataset/
 ↪train'
test_dir = '/content/drive/MyDrive/Deep Learning - datasets/alzheimer dataset/
 ↪test'

# Create the ImageDataGenerator object for data augmentation and preprocessing
train_datagen = ImageDataGenerator(
    rescale = 1./255,
    shear_range = 0.4,
    zoom_range = 0.2,
    width_shift_range = 0.2,
    height_shift_range = 0.2,
    rotation_range = 90,
    horizontal_flip = True,
    fill_mode = 'nearest'
)
test_datagen = ImageDataGenerator(rescale = 1./255)

# Load the images and labels, ensuring grayscale (1 color channel)

train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size = (128, 128),      # Resize the images to 128 x 128
    color_mode = 'grayscale',      # we have not used batch_size, so keras by␣
 ↪default chooses batch_size = 32
    class_mode = 'categorical'
```

```
)

test_generator = test_datagen.flow_from_directory(
    test_dir,
    target_size = (128,128),
    color_mode = 'grayscale',     # Load as grayscale
    class_mode = 'categorical',
)
```

MODEL BUILDING

VGG16 Original

[ ]:
```
from keras.models import Sequential
from keras.layers import Conv2D
from keras.layers import Flatten
from keras.layers import MaxPooling2D
from keras.layers import Dense
import numpy as np
```

[23]:
```
# VGG-16 style model implemented as a Sequential model
# Matches the original VGG-16 block structure shown in your snippet

input_shape = (128, 128,1)  # change if you need different input size
num_classes = 4  # change to the number of output classes you need

# in original model the no. of output classes is 1000

model = Sequential()

# Block 1
model.add(Conv2D(filters=64, kernel_size=(3, 3), padding='same',␣
 ↪activation='relu', input_shape=input_shape))
model.add(Conv2D(filters=64, kernel_size=(3, 3), padding='same',␣
 ↪activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2), strides=(2, 2)))

# Block 2
model.add(Conv2D(filters=128, kernel_size=(3, 3), padding='same',␣
 ↪activation='relu'))
model.add(Conv2D(filters=128, kernel_size=(3, 3), padding='same',␣
 ↪activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2), strides=(2, 2)))

# Block 3
model.add(Conv2D(filters=256, kernel_size=(3, 3), padding='same',␣
 ↪activation='relu'))
```

```
model.add(Conv2D(filters=256, kernel_size=(3, 3), padding='same',␣
 ↪activation='relu'))
model.add(Conv2D(filters=256, kernel_size=(3, 3), padding='same',␣
 ↪activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2), strides=(2, 2)))

# Block 4
model.add(Conv2D(filters=512, kernel_size=(3, 3), padding='same',␣
 ↪activation='relu'))
model.add(Conv2D(filters=512, kernel_size=(3, 3), padding='same',␣
 ↪activation='relu'))
model.add(Conv2D(filters=512, kernel_size=(3, 3), padding='same',␣
 ↪activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2), strides=(2, 2)))

# Block 5
model.add(Conv2D(filters=512, kernel_size=(3, 3), padding='same',␣
 ↪activation='relu'))
model.add(Conv2D(filters=512, kernel_size=(3, 3), padding='same',␣
 ↪activation='relu'))
model.add(Conv2D(filters=512, kernel_size=(3, 3), padding='same',␣
 ↪activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2), strides=(2, 2)))

# Classification head
model.add(Flatten())
model.add(Dense(units=4096, activation='relu'))
model.add(Dense(units=4096, activation='relu'))
model.add(Dense(units=num_classes, activation='softmax'))

# Compile (use appropriate loss for your labels; if binary labels use␣
 ↪'categorical_crossentropy' only when one-hot encoded)
model.compile(optimizer='adam', loss='categorical_crossentropy',␣
 ↪metrics=['accuracy'])

# Display summary
model.summary()
```

```
Model: "sequential_6"
```

Layer (type)	Output Shape	Param #
conv2d_72 (Conv2D)	(None, 128, 128, 64)	640
conv2d_73 (Conv2D)	(None, 128, 128, 64)	36,928

max_pooling2d_27 (MaxPooling2D)	(None, 64, 64, 64)	0
conv2d_74 (Conv2D)	(None, 64, 64, 128)	73,856
conv2d_75 (Conv2D)	(None, 64, 64, 128)	147,584
max_pooling2d_28 (MaxPooling2D)	(None, 32, 32, 128)	0
conv2d_76 (Conv2D)	(None, 32, 32, 256)	295,168
conv2d_77 (Conv2D)	(None, 32, 32, 256)	590,080
conv2d_78 (Conv2D)	(None, 32, 32, 256)	590,080
max_pooling2d_29 (MaxPooling2D)	(None, 16, 16, 256)	0
conv2d_79 (Conv2D)	(None, 16, 16, 512)	1,180,160
conv2d_80 (Conv2D)	(None, 16, 16, 512)	2,359,808
conv2d_81 (Conv2D)	(None, 16, 16, 512)	2,359,808
max_pooling2d_30 (MaxPooling2D)	(None, 8, 8, 512)	0
conv2d_82 (Conv2D)	(None, 8, 8, 512)	2,359,808
conv2d_83 (Conv2D)	(None, 8, 8, 512)	2,359,808
conv2d_84 (Conv2D)	(None, 8, 8, 512)	2,359,808
max_pooling2d_31 (MaxPooling2D)	(None, 4, 4, 512)	0
flatten_5 (Flatten)	(None, 8192)	0
dense_17 (Dense)	(None, 4096)	33,558,528
dense_18 (Dense)	(None, 4096)	16,781,312
dense_19 (Dense)	(None, 4)	16,388

Total params: 65,069,764 (248.22 MB)

Trainable params: 65,069,764 (248.22 MB)

```
Non-trainable params: 0 (0.00 B)
```

[ ]:
```
# Use 50 Epochs, training only 1 due to lack of access to computation
model.fit(train_generator,epochs=1)
```

[ ]:
```
# Evaluate the model on the test data
loss, accuracy_vgg = model.evaluate(test_generator)

# Print the accuracy
print(f"Test Accuracy of VGG16 Original: {accuracy_vgg}")
```

We can add BatchNorm + Dropout + smaller Dense layers.

For example:

[24]:
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense,␣
 ↪Dropout, BatchNormalization

model = Sequential()

# Block 1
model.add(Conv2D(64, (3,3), padding="same", activation="relu",␣
 ↪input_shape=(128,128,1)))
model.add(BatchNormalization())
model.add(Conv2D(64, (3,3), padding="same", activation="relu"))
model.add(BatchNormalization())
model.add(MaxPooling2D(pool_size=(2,2), strides=(2,2)))

# Block 2
model.add(Conv2D(128, (3,3), padding="same", activation="relu"))
model.add(BatchNormalization())
model.add(Conv2D(128, (3,3), padding="same", activation="relu"))
model.add(BatchNormalization())
model.add(MaxPooling2D(pool_size=(2,2), strides=(2,2)))

# Block 3
model.add(Conv2D(256, (3,3), padding="same", activation="relu"))
model.add(BatchNormalization())
model.add(Conv2D(256, (3,3), padding="same", activation="relu"))
model.add(BatchNormalization())
model.add(Conv2D(256, (3,3), padding="same", activation="relu"))
model.add(BatchNormalization())
model.add(MaxPooling2D(pool_size=(2,2), strides=(2,2)))

# Block 4 ( drop one of the 512 blocks if data is small)
model.add(Conv2D(512, (3,3), padding="same", activation="relu"))
```

```
model.add(BatchNormalization())
model.add(Conv2D(512, (3,3), padding="same", activation="relu"))
model.add(BatchNormalization())
model.add(Conv2D(512, (3,3), padding="same", activation="relu"))
model.add(BatchNormalization())
model.add(MaxPooling2D(pool_size=(2,2), strides=(2,2)))

# Global part
model.add(Flatten())
model.add(Dense(1024, activation="relu"))
model.add(Dropout(0.5))
model.add(Dense(512, activation="relu"))
model.add(Dropout(0.5))
model.add(Dense(4, activation="softmax"))
```

Option 2:

Use GlobalAveragePooling2D instead of Flatten

Option 3:

Consider transfer learning (especially if data is limited)

[ ]:
```
from tensorflow.keras.layers import GlobalAveragePooling2D

# ... conv blocks as before ...

model.add(GlobalAveragePooling2D())
model.add(Dense(256, activation="relu"))
model.add(Dropout(0.5))
model.add(Dense(4, activation="softmax"))
```

How to use GlobalAveragePooling:

[25]:
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, BatchNormalization
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense, Dropout

model = Sequential()

model.add(Conv2D(32, (3,3), activation='relu', padding='same',␣
 ↪input_shape=(128,128,1)))
model.add(BatchNormalization())
model.add(MaxPooling2D((2,2)))

model.add(Conv2D(64, (3,3), activation='relu', padding='same'))
model.add(BatchNormalization())
model.add(MaxPooling2D((2,2)))
```

```
model.add(Conv2D(128, (3,3), activation='relu', padding='same'))
model.add(BatchNormalization())
model.add(MaxPooling2D((2,2)))

model.add(GlobalAveragePooling2D())
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(4, activation='softmax'))
```

VGG 19 original

[26]:
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
```

[27]:
```
model = Sequential()

model.add(Conv2D(filters = 64, kernel_size=(3,3), strides = 1, padding='same',␣
 ↪input_shape = (128,128,1), activation = 'relu'))
model.add(Conv2D(filters = 64, kernel_size=(3,3),strides = 1, padding='same',␣
 ↪activation = 'relu'))
model.add(MaxPooling2D(pool_size = (2,2),strides = 2))

model.add(Conv2D(filters = 128, kernel_size=(3,3),strides = 1, padding='same',␣
 ↪activation = 'relu'))
model.add(Conv2D(filters = 128, kernel_size=(3,3),strides = 1, padding='same',␣
 ↪activation = 'relu'))
model.add(MaxPooling2D(pool_size = (2,2),strides = 2))

model.add(Conv2D(filters = 256, kernel_size=(3,3),strides = 1, padding='same',␣
 ↪activation = 'relu'))
model.add(Conv2D(filters = 256, kernel_size=(3,3),strides = 1, padding='same',␣
 ↪activation = 'relu'))
model.add(Conv2D(filters = 256, kernel_size=(3,3),strides = 1, padding='same',␣
 ↪activation = 'relu'))
model.add(Conv2D(filters = 256, kernel_size=(3,3),strides = 1, padding='same',␣
 ↪activation = 'relu'))
model.add(MaxPooling2D(pool_size = (2,2),strides = 2))

model.add(Conv2D(filters = 512, kernel_size=(3,3),strides = 1, padding='same',␣
 ↪activation = 'relu'))
model.add(Conv2D(filters = 512, kernel_size=(3,3),strides = 1, padding='same',␣
 ↪activation = 'relu'))
model.add(Conv2D(filters = 512, kernel_size=(3,3),strides = 1, padding='same',␣
 ↪activation = 'relu'))
model.add(Conv2D(filters = 512, kernel_size=(3,3),strides = 1, padding='same',␣
 ↪activation = 'relu'))
```

```
model.add(MaxPooling2D(pool_size = (2,2),strides = 2))
model.add(Conv2D(filters = 512, kernel_size=(3,3),strides = 1, padding='same',␣
 ↪activation = 'relu'))
model.add(Conv2D(filters = 512, kernel_size=(3,3),strides = 1, padding='same',␣
 ↪activation = 'relu'))
model.add(Conv2D(filters = 512, kernel_size=(3,3),strides = 1, padding='same',␣
 ↪activation = 'relu'))
model.add(Conv2D(filters = 512, kernel_size=(3,3),strides = 1, padding='same',␣
 ↪activation = 'relu'))

model.add(MaxPooling2D(pool_size = (2,2),strides = 2))
model.add(Flatten())
model.add(Dense(units = 4096,activation = 'relu'))
model.add(Dense(units = 4096,activation = 'relu'))
model.add(Dense(units = 4,activation = 'softmax'))

# Compile (use appropriate loss for your labels; if binary labels use␣
 ↪'categorical_crossentropy' only when one-hot encoded)
model.compile(optimizer='adam', loss='categorical_crossentropy',␣
 ↪metrics=['accuracy'])

# Display summary
model.summary()
```

```
Model: "sequential_9"
```

Layer (type)	Output Shape	Param #
conv2d_98 (Conv2D)	(None, 128, 128, 64)	640
conv2d_99 (Conv2D)	(None, 128, 128, 64)	36,928
max_pooling2d_39 (MaxPooling2D)	(None, 64, 64, 64)	0
conv2d_100 (Conv2D)	(None, 64, 64, 128)	73,856
conv2d_101 (Conv2D)	(None, 64, 64, 128)	147,584
max_pooling2d_40 (MaxPooling2D)	(None, 32, 32, 128)	0
conv2d_102 (Conv2D)	(None, 32, 32, 256)	295,168
conv2d_103 (Conv2D)	(None, 32, 32, 256)	590,080
conv2d_104 (Conv2D)	(None, 32, 32, 256)	590,080

```
conv2d_105 (Conv2D)              (None, 32, 32, 256)          590,080
max_pooling2d_41 (MaxPooling2D)  (None, 16, 16, 256)              0
conv2d_106 (Conv2D)              (None, 16, 16, 512)        1,180,160
conv2d_107 (Conv2D)              (None, 16, 16, 512)        2,359,808
conv2d_108 (Conv2D)              (None, 16, 16, 512)        2,359,808
conv2d_109 (Conv2D)              (None, 16, 16, 512)        2,359,808
max_pooling2d_42 (MaxPooling2D)  (None, 8, 8, 512)                0
conv2d_110 (Conv2D)              (None, 8, 8, 512)          2,359,808
conv2d_111 (Conv2D)              (None, 8, 8, 512)          2,359,808
conv2d_112 (Conv2D)              (None, 8, 8, 512)          2,359,808
conv2d_113 (Conv2D)              (None, 8, 8, 512)          2,359,808
max_pooling2d_43 (MaxPooling2D)  (None, 4, 4, 512)                0
flatten_7 (Flatten)              (None, 8192)                     0
dense_25 (Dense)                 (None, 4096)            33,558,528
dense_26 (Dense)                 (None, 4096)            16,781,312
dense_27 (Dense)                 (None, 4)                   16,388

Total params: 70,379,460 (268.48 MB)

Trainable params: 70,379,460 (268.48 MB)

Non-trainable params: 0 (0.00 B)
```

[ ]:
```
# Use 50 Epochs, training only 1 due to lack of access to computation
model.fit(train_generator,epochs=1)
```

[ ]:
```
# Evaluate the model on the test data
loss, accuracy_vgg19 = model.evaluate(test_generator)
```

```
# Print the accuracy
print(f"Test Accuracy of VGG19 Original: {accuracy_vgg19}")
```